

第8章 不相交集

建议学时:3-4 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**等价关系**三性质与"动态"等价问题为何要求在线回答
- 掌握树形表示:每集合一棵树,根是**代表元**,一个 `parent` 列表即可实现
- 完成 C 数组到 Python 列表的迁移(`list(range(n))` 一行初始化),写出朴素 `find / union`,踩一遍"链退化"
- 掌握**按大小合并**、**按秩合并**(树高 $O(\log n)$)与**路径压缩**(均摊近 $O(1)$),并避开 Python 递归深度的坑
- 学完能用并查集生成随机迷宫、统计连通分量,为第9章 Kruskal 铺路

💡 从 C 到 Python:本章主题如何迁移

本章是全书**代码量最小**、**思想密度最高**的一章:完整实现约 30 行,却支撑着网络连通、迷宫生成、第9章 Kruskal 算法等大应用。先给结论:不相交集(并查集)只做一件事——维护一堆集合,支持"查归属"与"合并"两种操作。

在 C 里你天然会想到笨办法:开 `mark[N]` 数组记录每个元素属于第几组,合并时遍历整个数组把所有第 `b` 组的元素改成第 `a` 组。能工作,但每次合并 $O(n)$,合并 n 次就是 $O(n^2)$ 。本章的树形表示把合并降为"改一个指针"——"用树的结构信息换时间"的经典一步。

Python 的迁移点:① C 的 `int parent[N]` 变成 `parent = list(range(n))`,初始化天然一行;② C 的裸数组函数传参变成类方法, `parent` 作为属性随对象走;③ 唯一要警惕的是**递归深度**:C 的递归 `find` 在 Python 里会撞上 `RecursionError` (默认限制约 1000 层),所以本章推荐迭代版 `find`。Python 标准库同样没有并查集,**手写就是全部**——每一行都要理解到位。

本章路线:先定义问题(\$8.1),写朴素实现踩一遍退化的坑(\$8.2),依次加两种优化(\$8.3-\$8.4),看清复杂度如何摊还到近 $O(1)$ (\$8.5),最后在迷宫与连通分量中验证(\$8.6-\$8.7)。

📦 前置知识自查

数学前置:等价关系三性质(自反、对称、传递);对数(第2章)——按大小合并的高度证明用 $O(\log n)$ 。

C 语言前置:`int` 数组、`for` 循环初始化、递归函数(\$8.4 对照 C 的递归 `find`)。

依赖的前面章节:第3章 Python 列表 `list`(存放 `parent` 数组)、第4章树的父指针表示(`parent` 数组正是"树"的另一种存法)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 \$8.1-\$8.2,跑通朴素实现,手工模拟"链退化"
2. **第二阶段(1.5 小时)**:读 \$8.3-\$8.4,实现按秩合并与迭代版路径压缩
3. **第三阶段(1 小时)**:读 \$8.5-\$8.7,理解摊还直觉,用并查集生成迷宫
4. **第四阶段(实验,1 小时)**:完成章末实验:迷宫生成 + 连通分量统计 + 性能对比
5. **验收标准**:能默写 20 行内的完整并查集,并说出三种配置的复杂度

本章知识导图

- §8.1 等价关系与动态等价问题:三性质、find/union、为何"动态"
- §8.2 基本实现:树表示与朴素 union/find(根=代表元、链退化)
- §8.3 按大小合并与按秩合并:size/rank 数组、树高 $O(\log n)$ 的证明
- §8.4 路径压缩:递归与迭代两写法、压扁树的机制、叠加效果
- §8.5 复杂度:摊还近 $O(1)$ 的直觉、反阿克曼函数 $\alpha(n)$
- §8.6 应用:迷宫生成、连通分量统计、Kruskal 预告、替代方案对比
- §8.7 实现要点与调试:初始化、先 find 到根、秩不失效、打印 parent

§8.1 等价关系与动态等价问题

8.1.1 等价关系:三性质

等价关系(equivalence relation)是定义在集合上的二元关系,满足三条性质:① **自反**: $a \equiv a$;② **对称**: $a \equiv b$ 则 $b \equiv a$;③ **传递**: $a \equiv b$ 且 $b \equiv c$, 则 $a \equiv c$ 。三条缺一不可。等价关系把集合划分成若干**等价类**:互相等价的元素属于同一类——这些类互不相交,正是"不相交"一词的来历。

例子:整数"相等"、整数"模 m 同余"、电路里"两点由导线连通",都是等价关系。再看**反例**:中学的"同班同学"——自反、对称都满足,但不**传递**:甲、乙同班,乙、丙同班,甲、丙未必同班。它不是等价关系,不能用并查集建模"全班连通"。

8.1.2 动态等价问题:union 与 find

给定 N 个元素,初始时每个元素自成一个集合。支持两种操作:

- **find(x)**:返回 x 所在集合的名字——用该集合的**代表元**(representative)当名字;
- **union(a, b)**:把 a 、 b 所在的两个集合合并成一个(若本就在同一集合,什么都不做)。

注意"动态"二字的分量:若所有关系一次性给定再问"1 号和 5 号通不通",那是**静态**问题,遍历一遍算出全部等价类即可(第9章的 DFS 能做)。实际场景中 **查询与合并交错到来**:接入一根新网线(合并),立刻有人问两台机器通不通(find),接着又接新线..... 每一步都必须在当下即时回答,不能等收集完所有操作再算。这正是"动态等价问题"的在线(on-line)性质,也是必须专门设计数据结构的理由。

8.1.3 应用预告

后面你会反复见到它:① **迷宫生成**:随机拆墙但保证任意两格连通(§8.6 详解,章末实验);② **Kruskal 最小生成树**(第9章):按边权从小到大加边,用 **find** 判断两端是否已连通——并查集是它的灵魂;③ **工程应用**:网络断点检测、图像连通域标记。记住一句话:凡是"合并集合 + 查询归属"成对出现的问题,先想并查集。

§8.2 基本实现:树表示与朴素 union/find

8.2.1 树表示:根就是代表元

怎么表示"一组元素属于同一集合"?答案是**树**:每个集合一棵树,树的**根是代表元**——**find(x)** 沿父链上溯到根并返回根,集合的名字即根;合并时让一棵树的根指向另一棵树的根,只改一个指针。这是第4章"父指针表示法"的直接应用:**parent[i]** 是结点 i 的父结点,根满足 **parent[root] == root** ("我是根"的哨兵约定)。

8.2.2 从 C 的数组到 Python 的列表

C 里写 `int parent[N]`; Python 换成 `list, list(range(n))` 一行完成创建与初始化:

C 的写法	Python 的写法
<code>int parent[N];</code> 长度写死	<code>parent = [0] * n</code> 或 <code>list(range(n))</code>
<code>for (i=0; i<N; ++i) parent[i] = i;</code>	<code>parent = list(range(n))</code> 一行初始化
越界访问 = 未定义行为	越界抛 <code>IndexError</code> , 立即暴露 bug
函数传参要带长度 N	<code>len(parent)</code> 自带长度

示例 8-1 朴素实现: find 沿父链上溯, union 让根指向根

```
# parent[i] == i 表示 i 是根; 其余 parent[i] 是 i 的父结点
parent = list(range(8))      # 一行初始化: 每个元素都是自己的根

def find(x):                 # 朴素 find: 沿父链上溯到根
    while parent[x] != x:
        x = parent[x]
    return x

def union_sets(a, b):        # 朴素 union: 让 a 的根指向 b 的根
    ra, rb = find(a), find(b)
    if ra != rb:
        parent[ra] = rb

union_sets(1, 2)
union_sets(3, 4)
union_sets(2, 3)             # 现在 {1,2,3,4} 在同一集合
# find(1) -> find(2) -> find(3) -> find(4): 一路到根 4
print(find(4))
```

★ 重点: 三个必须想清楚的点

① `union` 前必须先 `find`: 合并的是“两个集合”不是“两个元素”, 拿根来挂; 只有根能当代表元, 挂普通结点会破坏树。② `parent` 里存的是“父结点”不是“集合编号”: 集合没有全局编号, 根的名字就是集合的名字, `find` 必须走到根才能回答归属。③ 初始化绝不能省: `parent = list(range(n))` 让每个元素自成一棵单结点树, 这是推理的地基。

8.2.3 朴素实现的致命伤: 链退化

⚠ 误区 1: 朴素 union 会把树连成一条链

依次执行 `union(0,1)`, `union(1,2)`, ..., `union(n-2, n-1)`: 第一次让 0 指向 1, 第二次让 1 指向 2.....`parent` 变成 `0->1, 1->2, ..., n-2->n-1`, 树被拉成一条 n 结点的链。`find(0)` 要走 $O(n)$ 步, 做 n 次 `find` 就是 $O(n^2)$ 。链 = 树高的灾难: 树形表示的全部好处都建立在“树要矮”上, §8.3 的两种合并策略正是从“让树长不高”入手。

§8.3 按大小合并与按秩合并

8.3.1 按大小合并:小树并入大树

按大小合并(union by size)的规则一句话:总是把结点数少的树挂到结点数多的树下。为此维护 `size` 列表: `size[root]` 是根所在树的结点数(只在根上有效),合并时小根指向大根并累加。

示例 8-2 按大小合并:size 只在根上有效

```
# sz[root] = 根所在树的结点数(只在根上有效)
parent = list(range(8))
sz = [1] * 8          # 初始每棵树 1 个结点

def find(x):           # 朴素 find 不变
    while parent[x] != x:
        x = parent[x]
    return x

def union_by_size(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return         # 已在同一集合
    if sz[ra] < sz[rb]: # 小树并入大树
        ra, rb = rb, ra
    parent[rb] = ra     # rb 不再是根,它的 sz 作废
    sz[ra] += sz[rb]
```

8.3.2 按秩合并:浅树并入深树

按秩合并(union by rank)是"按大小"的孪生兄弟:目标都是**别让树长高**。它维护 `rank` 列表: `rank[root]` 是树高的上界(初始 0)。合并时秩小的树挂到秩大的树下;秩相同则任选一树做根并把秩加一。实现更短,且"上界"语义给 §8.4 路径压缩更大自由度。

示例 8-3 按秩合并:浅树挂到深树

```
# rank[root] = 树高的上界(只在根上有效),初始 0
parent = list(range(8))
rank = [0] * 8

def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

def union_by_rank(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return
    if rank[ra] < rank[rb]: # 浅树挂到深树
        ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]: # 两棵同秩树合并
        rank[ra] += 1      # 新树秩 +1
```

8.3.3 为什么树高是 $O(\log n)$

两种策略是同一句证明: 一个结点深度每增加 1, 它所在树的规模至少翻一倍。x 深度增加只发生在“它的树被并入另一棵树”时: 按大小合并规定小树并入大树, 新树大小 $\geq 2 \times$ 旧树; 按秩合并规定浅树并入深树, 而秩 r 的树至少 2^r 个结点(归纳可证)。故 x 深度最多增加 $\log_2 n$ 次, 树高 $\leq \log_2 n$, find 保证 $O(\log n)$ 。n = 10^6 时 $\log_2 n \approx 20$, 链却是 10^6 。

策略	维护什么	合并规则	何时 +1	附带能力
按大小合并	size[root] = 结点数	小树并入大树	新根 size 累加	可查询集合大小
按秩合并	rank[root] = 高度上界	浅树并入深树	两秩相等时 +1	与路径压缩配合更顺

★ 重点: 两种策略等价, 选一个记住

两者都保证树高 $O(\log n)$, 复杂度相同。按秩合并代码更短, 且“秩是上界”的语义与路径压缩天然兼容(见 §8.7.2); 按大小合并的额外好处是能回答“集合有几个元素”。本章正文与实验采用按秩合并, 换成 size 只需三行。

§8.4 路径压缩

8.4.1 思想: 把树压扁

按大小/按秩合并把树高控制在 $O(\log n)$, 但还能更狠: 路径压缩(path compression)在 find 的路途中动手——既然已经走到根, 就把沿途每个结点直接挂到根下面, find(x) 之后 x 到根的距离变成 1, 整条路径被压成“菊花状”。压缩只发生在 find 上, union 调用 find 时自动受益——一次 find 的代价, 换路径上所有结点后续 find 的 $O(1)$ 。

8.4.2 递归实现与迭代实现

示例 8-4 路径压缩: 递归与迭代两种写法

```
# 递归版: 返回根, 回溯时把沿途结点挂到根
def find_rec(x):
    if parent[x] != x:
        parent[x] = find_rec(parent[x])
    return parent[x]

# 迭代版(两趟): 先找根, 再沿原路压缩
def find_iter(x):
    root = x
    while parent[root] != root: # 第一趟: 上溯找根
        root = parent[root]
    while parent[x] != x: # 第二趟: 沿途结点挂到根
        nxt = parent[x]
        parent[x] = root
        x = nxt
    return root
```

递归版三行, 但 Python 里有一个**必须正视的隐患**: 递归深度等于树高, 而 Python 默认递归上限只有约 1000 层。按秩合并保证树高 $O(\log n)$ 时完全安全; 若只压缩不按秩(很多竞赛模板这样), 深链上 find_rec(0) 会直接抛 RecursionError。调 sys.setrecursionlimit 能缓解, 但治标不治本。本章的工程结论: Python 版一律用迭代两趟版, 无递归深度风险——这也是 Python 与 C 的重要差异(见 §8.7.2)。

C 的写法(递归上溯)	Python 的写法(迭代两趟)
<pre>int f(int x){ return p[x]==x ? x : f(p[x]); }</pre>	<pre>while parent[root] != root: root = parent[root] 第一趟找根</pre>
递归深度 = 树高,深链会爆栈	深链递归会抛 RecursionError(默认约 1000 层)
修改发生在回溯期,代码短	第二趟显式压缩,无递归深度风险

⚠ 误区 2:Python 递归 find 撞上 RecursionError

路径压缩 + 按秩合并时递归没问题;但只压缩不按秩时树高可达 n , `find_rec(0)` 递归深度就是 n , n 超过约 1000 立即抛 `RecursionError: maximum recursion depth exceeded`。更隐蔽的坑是忘了用压缩版:写了 `find_iter` 却仍调用朴素 `find`, 压缩永不发生。检查:连续 `find` 两次同一结点,看路径是否变短。

8.4.3 与按秩合并叠加:近 $O(1)$

按秩合并让树"长不高"(高度 $O(\log n)$), 路径压缩让树"越来越扁", 叠加后, 并查集的操作复杂度被压到"均摊近 $O(1)$ "——精确上界是反阿克曼函数 $\alpha(n)$, 见 §8.5.3。这是本数据结构的最终形态: **union = 两次 find + 一次指针修改**, **find $\approx O(1)$** , 请务必默写。

🚀 实现建议:迭代两趟的写法与验证

迭代两趟容易把变量搞混, 验证三点:① 第一趟后 `root` 必须是根 (`parent[root] == root`); ② 第二趟先保存 `nxt = parent[x]` 再改写 `parent[x]`, 否则覆盖后丢链; ③ 返回 `root` 而非 `x`。用幂等性自测:连续两次 `find(x)`, 第二次必然一步到根。

§8.5 复杂度:摊还近 $O(1)$

8.5.1 三种配置,三个档次

配置	单次 find 最坏	m 次操作总代价
朴素实现(无优化)	$O(n)$ (链)	$O(mn)$
仅按大小/按秩合并	$O(\log n)$	$O(m \log n)$
按秩合并 + 路径压缩	均摊近 $O(1)$ (单次最坏仍 $O(\log n)$)	$O(m \alpha(n))$, 实际当 $O(m)$ 用

读表注意"单次"与"总代价"之分:完整优化下**某一次** find 仍可能慢(遇到未压缩深链,一次性压扁),但"债"被后续 find 偿还——这就是**摊还**:把总代价摊到每次操作上,平均近 $O(1)$ 。

8.5.2 摊还近 $O(1)$ 的直觉

问: x 的 `parent` 会被改写几次? 压缩改写它时直接挂到根, 之后每次 find 一步到达; 只有"它所在的树又并入更大的树"时, 它才可能再被改写。按秩合并保证每次并入树至少翻倍, 故每结点最多被改写 $O(\log n)$ 次——这只是粗糙上界, "压扁"带来的加速使真实总量小得多: 1975 年 Tarjan 证明 m 次操作总代价 $O(m \alpha(n))$ 。工程上, 把并查集操作当作 $O(1)$ 用, 99.99% 的场景不会出错。

8.5.3 反阿克曼函数:为什么当常数用

反阿克曼函数 $\alpha(n)$ 是阿克曼函数(增长极快的递归函数)的反函数:使阿克曼函数达到 n 所需的最小自变量。它慢到难以想象: $\alpha(10^{80}) \leq 4 \sim 5$ ——"全宇宙原子数"级别的 n 也只让 α 走到 4、5。故 $O(m \alpha(n))$ 在实践中就是 $O(m)$:**每次 union/find 均摊近 $O(1)$, 当常数用**。这与第2章"log 几乎不增长"一脉相承—— α 是更极端的"几乎不增长"。

记号说明(本章约定)

$\alpha(n)$ 读作"反阿克曼",是分析意义上的上界;标准写法"均摊近 $O(1)$ ",严谨写法 $O(\alpha(n))$ 。"摊还" $\neq$ "平均":平均对随机输入取期望,摊还对**同一个操作序列**的总代价取平均——并查集的近 $O(1)$ 不依赖任何随机性假设。

§8.6 应用:迷宫生成与连通分量

8.6.1 迷宫生成:随机拆墙,并查集判连通

把 $w \times h$ 网格的每格看成一个集合(初始互不相交),收集所有相邻格子之间的墙,**随机打乱顺序**后依次尝试拆墙:两端**不连通**就拆掉并 union,已连通则留着,直到只剩一个集合。两个性质:

- **无环**:每面被拆的墙都连接两个原本不连通的区域,永远不会形成回路;
- **全连通**:结束时只有一个集合,任意两格都能到达。

共 $n = w \times h$ 个格子,成功拆墙恰好 $n-1$ 面(每拆一面分量减一,从 n 减到 1)。 $n-1$ 面墙、无环、全连通——正是**生成树**:迷宫即网格图的随机生成树,任意两格之间有且仅有一条路径。学术上叫"随机 Kruskal 迷宫",与第9章 Kruskal 最小生成树的唯一区别:墙的"权"是随机顺序。

示例 8-5 迷宫生成核心:拆墙循环(完整程序见章末实验)

```
# 格子编号 id = r*w + c;right/down 记录墙
def gen_maze(w, h, seed=2024):
    uf = UnionFind(w * h)          # 完整优化版并查集
    right = [[True] * w for _ in range(h)] # 右墙
    down = [[True] * w for _ in range(h)]  # 下墙
    walls = []
    for r in range(h):
        for c in range(w):
            if c + 1 < w:
                walls.append((r, c, r, c + 1)) # 右墙
            if r + 1 < h:
                walls.append((r, c, r + 1, c)) # 下墙
    random.seed(seed)
    random.shuffle(walls)           # 随机顺序拆墙
    cnt = w * h                     # 连通分量个数
    for r1, c1, r2, c2 in walls:
        if uf.union(r1 * w + c1, r2 * w + c2): # 两端不连通才拆
            cnt -= 1                        # 每拆一面墙,分量减一
            if c1 == c2:
                down[r1][c1] = False
            else:
                right[r1][c1] = False
    if cnt == 1:
        break                        # 全连通即停
    return right, down
```

8.6.2 连通分量统计

并查集自带连通分量信息:初始化时分量数为 n ;每次 `union` 成功(两端原本不连通)分量数减一——生成迷宫的同时就能报出"还有几个连通块",实验任务 3 正是记录这条下降曲线。若没维护计数,也可遍历全部元素数根: `find(i)` 即 i 的代表元,根相同即同一分量。前者 $O(1)/\text{次}$,后者 $O(n \alpha(n))$ 。

8.6.3 手写并查集 vs dict 标记法:优势场景

与本章开头提过的"笨办法"认真对比一次,才明白并查集赢在哪。Python 里"标记法"的天然载体是 dict:

示例 8-6 替代方案:用 dict 维护"元素 -> 集合编号"

```
# 替代方案:union 时把较小集合的所有元素逐个改编号
owner = {i: i for i in range(n)}      # 元素 -> 集合编号
def find_owner(x):
    return owner[x]                    # 查归属  $O(1)$ 

def union_owner(a, b):
    oa, ob = owner[a], owner[b]
    if oa == ob:
        return
    # 遍历全部元素把集合 ob 的成员改成 oa: $O(n)$ !
    for x, o in owner.items():
        if o == ob:
            owner[x] = oa
# 并查集:union 只改一个指针,均摊近  $O(1)$ ,差距  $n$  倍
```

C 的连通性标记法	Python 的并查集
<code>mark[N]</code> 记录归属,合并要遍历数组改编号, $O(n)$	<code>parent</code> 树形表示,合并改一个指针,均摊近 $O(1)$
查归属 $O(1)$,但合并贵	查归属(两次 <code>find</code>)与合并都近 $O(1)$
能直接枚举集合的全部成员	不支持枚举成员(需另存集合列表)
合并 m 次总代价 $O(mn)$	合并 m 次总代价 $O(m \alpha(n))$

手写 vs 替代:标准库没有并查集,两种方案都得手写。选择标准:频繁"合并 + 查归属"用并查集;需"枚举集合成员"或"合并极少"才考虑 dict 标记法。迷宫 10^6 格时,标记法每次拆墙扫描 10^6 个元素,并查集是常数步。这也解释了第9章 Kruskal 为何必须配并查集:换成标记法,排序的 $O(E \log E)$ 会被合并的 $O(E \vee)$ 吞掉,算法不可用。

§8.7 实现要点与调试

8.7.1 三个必须养成的习惯

- **初始化**用 `list(range(n))` (或 `parent[i] = i`):漏掉初始化,find 沿未定义值乱走甚至死循环,多半不报错,最危险。
- **合并前先 find 到根**:直接写 `parent[a] = b` 是"元素挂元素",树结构立刻损坏。
- **size/rank 只在根上维护**:非根结点的 size/rank 已过期,更新只能发生在合并后当选新根的结点上。

8.7.2 路径压缩不破坏秩的近似性

反直觉但正确的结论: `find` 做路径压缩时,不需要(也不应该)更新 `rank`。`rank` 是"树高的上界":压缩把树压矮,实际高度变小,旧 `rank` 仍是合法上界,只是"松"了——下次合并最多让树高略高于最优,不影响 $O(\log n)$ 与均摊近 $O(1)$ 。`find` 里更新 `rank` 反而白付代价。这就是"秩 = 上界"而非"精确高度":只承诺上界,压缩无损。

另一点 Python 特有:用 `sys.setrecursionlimit(10**6)` 硬撑深递归,会把解释器的 C 栈推向崩溃边缘(深递归在 CPython 里可能直接崩溃而非优雅报错)。所以再强调一次:Python 版 `find` 用迭代两趟,彻底绕开递归深度这个变量。

8.7.3 调试技巧:把结构画出来

示例 8-7 调试利器:打印 `parent` 列表

```
# 打印 parent 列表,观察树形态
def dump(parent):
    print(" ".join(f"{i}->{parent[i]}" for i in range(len(parent))))

# 验证"根性":每个元素的 find 结果必须是根
def check(parent):
    def find(x):
        while parent[x] != x:
            x = parent[x]
        return x
    for i in range(len(parent)):
        assert find(i) == parent[find(i)]

# 小数据验证:union_sets(0,1); union_sets(1,2); dump(parent)
# 输出 "0->1 1->2 2->2" 立刻看出 0-1-2 长链;压扁后 "0->2 1->2 2->2"。
```

调试三招

① 小数据手推: n 取 5~8,把每步 `union` 后的 `parent` 抄在纸上与 `dump` 对照;② 打印关键点:`find` 返回前打印"`x` 到根",压缩是否生效一目了然;③ 断言根性: `assert parent[find(i)] == find(i)`,把"结构被破坏"提前暴露为断言失败。

误区 3:初始化写成"全部指向 0"

把 `parent` 初始化成 `[0] * n` 而不是 `list(range(n))`:`find(3)` 走到 0 就停下,0 被误当所有集合的根,任何两个元素的 `find` 结果都相同——"全都连通"的假象,且不崩溃、不报错。排查口诀:初始化后立刻 `dump`,`parent` 必须等于下标。顺带一提, `[0] * n` 里放整数没有引用共享问题,但换成列表就会踩 Python 的"可变对象重复引用"坑,用 `[[False] * w for _ in range(h)]` 而不是 `[[False] * w] * h`。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
find	朴素实现(无优化)	$O(n)$	$O(n)$ (链状树)
union	朴素实现(无优化)	$O(n)$	$O(n)$
find	仅按大小合并	$O(\log n)$	$O(\log n)$
find / union	仅按秩合并	$O(\log n)$	$O(\log n)$
find	按秩合并 + 路径压缩	均摊近 $O(1)$ (严格 $O(\alpha(n))$)	
union	按秩合并 + 路径压缩	均摊近 $O(1)$ (两次 find + 一次指针修改)	
初始化	建 n 个单元素集合	$O(n)$	$O(n)$
递归版压缩 find	递归实现	均摊近 $O(1)$	深链递归抛 RecursionError(默认约 1000 层)
迭代版压缩 find	迭代实现(两趟)	均摊近 $O(1)$	均摊近 $O(1)$,无递归深度问题
求连通分量个数	遍历全部元素数根	$O(n \alpha(n))$	$O(n \alpha(n))$
维护连通分量数	union 成功时 count-1	$O(1)$	$O(1)$
迷宫生成	随机拆墙 + 并查集判连通	$O(E \alpha(V))$	$O(E \alpha(V))$
Kruskal 判连通	每条边两次 find + 一次 union	均摊近 $O(1)$ /边,总 $O(E \log E)$ (排序主导)	
反阿克曼函数 $\alpha(n)$	完整优化的单次操作上界	$\alpha(10^{80}) \leq 4 \sim 5$,当常数用	
dict 标记法	dict 维护集合编号	union $O(n)$ (遍历集合)	union $O(n)$

最小必会清单

- 能说出等价关系三性质,判断"同班同学""模 m 同余"是否等价关系
- 能说出 find / union 的语义,以及"动态"为何要求在线回答
- 能用 `list(range(n))` + 树形表示实现朴素并查集,解释"根 = 代表元"
- 能解释朴素 union 为何把树连成链,find 退化 $O(n)$
- 能实现按大小/按秩合并,说出"深度每 +1 树规模至少翻倍"的直觉
- 能写出路径压缩的递归版与迭代版,说出 Python 为何必须用迭代版
- 能说出三种配置的复杂度:无优化 $O(n)$ /仅按秩 $O(\log n)$ /完整优化均摊近 $O(1)$,及"当常数用"的理由
- 能用并查集生成随机迷宫,并调试:初始化、先 find 到根、rank 不失效、打印 parent

自测题

Q 1. "同班同学"关系满足等价关系的哪几条?缺哪条?为什么它导致"全班连通"不能用并查集建模?

✅ 参考答案

自反、对称满足,传递不满足:甲、乙同班且乙、丙同班,甲、丙可能不同班。三条不全,"同学"不能把集合划分成互不相交的等价类,故不能用并查集。

Q 2. 依次执行 `union(0,1)`, `union(1,2)`, ..., `union(n-2, n-1)` 后, `find(0)` 要走多少步?为什么是灾难?

✓ 参考答案

树变成链 $0 \rightarrow 1 \rightarrow \dots \rightarrow n-1$, `find(0)` 走 $n-1$ 步即 $O(n)$; n 次 `find` 总代价 $O(n^2)$, $n = 10^6$ 时是 10^{12} 步, 远超可行范围。这正是朴素实现必须优化的原因。

Q 3. 按大小合并为什么保证树高 $O(\log n)$? 给出直觉证明。

✓ 参考答案

x 深度每 $+1$, 必然是它所在的小树被并入一棵不小于自己的树(小树并入大树), 新树大小 $\geq 2 \times$ 旧树, 故 x 深度最多增加 $\log_2 n$ 次, 树高 $\leq \log_2 n$ 。

Q 4. 路径压缩为何能让后续 `find` 变快? Python 里递归版与迭代版各自的风险是什么?

✓ 参考答案

一次 `find` 把沿途所有结点挂到根, 之后它们再 `find` 都是一步到达——"一次的代价, 换一路的 $O(1)$ "。递归版代码短, 但递归深度 = 树高, 深链会抛 `RecursionError` (默认约 1000 层), 调 `setrecursionlimit` 也只是把问题推迟; 迭代版无递归深度风险, 工程更稳。

Q 5. 迷宫生成中, 为何拆墙数恰好等于格子数 - 1? 若拆墙时两端已连通会怎样?

✓ 参考答案

每拆一面墙必连通两个原本不相交的集合, 分量数恰好减一, 从 n 减到 1 共拆 $n-1$ 面。若两端已连通还拆墙会形成回路, 破坏"恰有一条路", 必须用 `find` 跳过。 $n-1$ 面、无环、全连通 = 生成树。

章末实验题: 迷宫生成与连通分量统计

实验 8: 迷宫生成与连通分量统计(并查集综合应用)

实验目标: 实现完整优化并查集(按秩合并 + 路径压缩), 生成随机 ASCII 迷宫, 统计拆墙中连通分量的变化, 实测朴素版与优化版性能差距。

实验要求:

- 任务 1: 实现并查集类: `find` (迭代两趟路径压缩) + `union` (按秩合并), 维护 `count` (知识点: §8.3 按秩合并、§8.4 路径压缩)
- 任务 2: 对 $w \times h$ 网格随机拆墙, 两端不连通才拆, 直到全连通; 输出 ASCII 迷宫(知识点: §8.6 迷宫生成、§8.2 树表示)
- 任务 3: 记录拆墙中连通分量数量的变化, 在尝试 25%/50%/75%/100% 处输出关键点, 验证"每拆一面墙分量减一"(知识点: §8.6.2 连通分量统计)
- 任务 4: 实现朴素版(无优化)做同一操作序列, 对比耗时, 体会 $O(n^2)$ 与近 $O(1)$ 的差距(知识点: §8.5 复杂度)

实验环境: Python 3.8+。运行: `python lab.py`。

第一步 写并查集类: `parent = list(range(n))`, `rank = [0] * n`, `count = n`; `find` 写迭代两趟版, `union` 按秩合并, 成功时 `count-1`

第二步 选墙: 右墙 $(r, c)-(r, c+1)$ 、下墙 $(r, c)-(r+1, c)$, `random.seed(seed)` + `random.shuffle` 固定种子随机化

第三步 墙循环中维护 `record`, 在 `idx` 达到墙总数的 1/4、1/2、3/4、全部时记录 (`idx`, `count`)

第四步 渲染按"顶/底横线 + 格子行 + 中间横线"拼字符串, 行宽一致(每格 3 字符)

第五步 任务 4 用同一操作序列(依次 `union(i, i+1)` 再全部 `find`)跑朴素版与优化版, `time.perf_counter()` 计时; 朴素版 `union` 的代价会延迟到 `find` 阶段爆发, 是观察"摊还"的绝佳素材

✅ 参考答案(完整可运行程序,约 143 行,分三段)

```
# lab.py — 迷宫生成与连通分量统计(第一段:并查集与朴素版)
# 运行:python lab.py
import random
import time

# ----- 任务 1:并查集(按秩合并 + 路径压缩) -----
class UnionFind:
    """按秩合并 + 路径压缩的并查集,parent[i] == i 表示 i 是根。"""

    def __init__(self, n):
        self.parent = list(range(n)) # parent[i] = i 表示 i 是根
        self.rank = [0] * n         # 秩:树高的上界,只在根上有意义
        self.count = n               # 当前连通分量个数

    def find(self, x):
        """迭代版路径压缩:先上溯找根,再沿原路把沿途结点挂到根。"""
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != x: # 第二趟:逐个压缩
            nxt = self.parent[x]
            self.parent[x] = root
            x = nxt
        return root

    def union(self, a, b):
        """合并 a、b 所在集合;合并成功返回 True,已在同一集合返回 False。"""
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]: # 浅树挂到深树
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1 # 两棵同秩树合并,新树秩 +1
        self.count -= 1       # 成功合并一次,连通分量减 1
        return True

# ----- 任务 4 对比用:朴素版(无任何优化) -----
class NaiveUF:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a != b:
            self.parent[a] = b # 固定让 a 挂到 b 下,容易形成长链
```

lab.py(第二段:迷宫生成与渲染)

```
# ----- 任务 2/3:随机迷宫生成 -----
def gen_maze(w, h, seed=2024):
    uf = UnionFind(w * h)
    right = [[True] * w for _ in range(h)] # 右墙
    down = [[True] * w for _ in range(h)] # 下墙
    walls = []
    for r in range(h):
        for c in range(w):
            if c + 1 < w:
                walls.append((r, c, r, c + 1)) # 右墙,连接 (r,c) 与 (r,c+1)
            if r + 1 < h:
                walls.append((r, c, r + 1, c)) # 下墙,连接 (r,c) 与 (r+1,c)
    random.seed(seed)
    random.shuffle(walls)
    record = [] # 任务 3:连通分量变化采样
    key = [len(walls) // 4, len(walls) // 2, len(walls) * 3 // 4]
    ki = 0
    for idx, (r1, c1, r2, c2) in enumerate(walls, 1):
        if uf.union(r1 * w + c1, r2 * w + c2): # 两端不连通才拆墙
            if c1 == c2:
                down[r1][c1] = False
            else:
                right[r1][c1] = False
        if ki < 3 and idx >= key[ki]:
            record.append((idx, uf.count))
            ki += 1
    record.append((len(walls), uf.count)) # 结束时必为 1
    return right, down, record

def render(w, h, right, down):
    """把迷宫画成 ASCII:每格 2 字符宽,"+" 是柱子,"--" 是横墙,"|" 是竖墙。"""
    lines = ["+" + "--+" * w] # 顶部横线(永不拆)
    for r in range(h):
        row = "|" # 最左竖墙(永不拆)
        for c in range(w):
            row += " " + ("|" if right[r][c] else " ")
        lines.append(row)
        if r == h - 1:
            lines.append("+ + --+" * w) # 底部横线(永不拆)
        else:
            line = "+"
            for c in range(w):
                line += "--" if down[r][c] else " "
                line += "+"
            lines.append(line)
    return "\n".join(lines)
```

lab.py(第三段:性能对比与主程序)


```

# ----- 任务 4:朴素版 vs 完整优化版性能对比 -----
def bench(n):
    ops = [(i, i + 1) for i in range(n - 1)] # 顺序合并:朴素版会连成长链
    t0 = time.perf_counter()
    naive = NaiveUF(n)
    for a, b in ops:
        naive.union(a, b) # 每次 union 都沿链走 O(n)
    t1 = time.perf_counter()
    s = 0
    for x in range(n):
        s += naive.find(x) # 链尾 find 走 O(n)
    t2 = time.perf_counter()
    opt = UnionFind(n)
    for a, b in ops:
        opt.union(a, b)
    t3 = time.perf_counter()
    for x in range(n):
        s += opt.find(x)
    t4 = time.perf_counter()
    ms = lambda a, b: (b - a) * 1000
    print(f"任务4: {n} 次 union(连成链)+ {n} 次 find(耗时对比):")
    print(f" 朴素版: union 阶段 {ms(t0, t1):.0f} ms, find 阶段 {ms(t1, t2):.0f} ms")
    print(f" 优化版: union 阶段 {ms(t2, t3):.0f} ms, find 阶段 {ms(t3, t4):.0f} ms")
    print(f" find 阶段加速比:约 {(t2 - t1) / (t4 - t3):.0f} 倍(检查值 s={s % 1000})")

def main():
    w, h = 6, 4
    right, down, record = gen_maze(w, h, seed=2024)
    print("=" * 44)
    print(f"随机迷宫 {w} 列 x {h} 行(种子 2024,共 {w * h} 个格子):")
    print(render(w, h, right, down))
    print()
    total = record[-1][0]
    print(f"任务3: 拆墙过程中连通分量数量的变化(共 {total} 面候选墙):")
    print(f" 初始: {w * h} 个连通分量")
    for tried, comp in record:
        print(f" 尝试 {tried:2d}/{total} 面墙后: 连通分量 {comp:2d}")
    print(f" 结束时连通分量为 1,迷宫全连通、无环(恰好拆 {w * h - 1} 面墙)。")
    print()
    bench(5000)

if __name__ == "__main__":
    main()

```

示例输出(本机 Python 3 实测):

=====

随机迷宫 6 列 x 4 行(种子 2024,共 24 个格子):

```
+---+---+---+---+
|           |     |
+ + + + + + +
| | | |   | |
+---+ +---+ +---+ +
|   | |   | |
+ + + + +---+ +
| |   | |   |
+---+---+---+---+
```

任务3: 拆墙过程中连通分量数量的变化(共 38 面候选墙):

初始: 24 个连通分量

尝试 9/38 面墙后: 连通分量 15

尝试 19/38 面墙后: 连通分量 6

尝试 28/38 面墙后: 连通分量 2

尝试 38/38 面墙后: 连通分量 1

结束时连通分量为 1,迷宫全连通、无环(恰好拆 23 面墙)。

任务4: 5000 次 union(连成链)+ 5000 次 find(耗时对比):

朴素版: union 阶段 1 ms, find 阶段 268 ms

优化版: union 阶段 1 ms, find 阶段 0 ms

find 阶段加速比:约 578 倍(检查值 s=0)

设计说明:① UnionFind 与 NaiveUF 共用同一操作序列,对比才公平——差的是增长阶($O(n^2)$ vs 近 $O(1)$),不是常数;② 朴素版 union 阶段并不慢(两版都是 1 ms):链的代价被"延迟"到 find 阶段爆发(268 ms vs 0 ms)——\$8.5 摊还思想的注脚:比算法要看操作序列的**总代价**;③ 任务 3 采样点取"尝试墙数"而非"成功拆墙数":拆墙有失败(两端已连通),曲线呈阶梯状,更能体现 find 判连通;④ 种子固定 2024,任何机器可复现;⑤ 固定挂法 `parent[a] = b` 刻意制造长链,暴露退化——现实中盲目选挂向也可能遭遇同样最坏输入。

下一章预告:第9章 图论算法——图的表示、拓扑排序、最短路径与最小生成树。Kruskal 将使用本章的不相交集:边按权排序后,find 判断两端是否已连通,union 把选中边连入生成树——两条核心操作正是本章主线。